ASDI Group Project: Comprehensive Discussion

Advanced Systems Design and Implementation

Document Organization

This comprehensive discussion consolidates all previous prompts and responses related to the ASDI Group Project into a single, well-organized document. The content is structured to guide student teams from project initiation through final deliverables, including the critical addition of actual customization/development and system demonstration.

Part 1: Project Overview and Objectives

1.1 Course Context

ASDI - Advanced Systems Design and Implementation is a capstone-level course that bridges the gap between high-level strategic architecture and hands-on implementation. The group project represents the culmination of all learning from Modules 1 through 6, requiring students to apply the four architectural domains—Business, Data, Application, and Technology—to design and implement a transformative solution for a hypothetical company.

Unlike individual activities where students worked on isolated components, this project requires teams to produce a cohesive, integrated architecture that addresses real business problems using modern technologies including polyglot persistence, APIs, and machine learning or artificial intelligence.

1.2 Core Project Requirements

Requirement	Description
Company Creation	Develop a hypothetical company with realistic IT asset management needs

Requirement	Description
Problem Identification	Document current pain points (minimum 10) with impact assessment
TO-BE Architecture	Design all four architectures based on team-agreed future state
Software Selection	Choose between customizing existing software or developing new system
Technology Integration	Incorporate polyglot persistence, APIs, and ML/AI
Instructor Approval	Seek approval before proceeding with customization or development
Implementation	Build working customizations or prototype
Demonstration	Present live or recorded demo of working system

1.3 Learning Objectives

Upon completing this project, students will be able to:

1. Model a system's landscape using Business, Data, Application, and Technology architectures
2. Visualize application architecture using the C4 model at all four levels
3. Justify and implement Polyglot Persistence strategies based on data requirements
4. Design and implement features that leverage AI/ML services

5. Navigate the transition from architecture design to implementation and deployment
6. Demonstrate a working system that implements architectural designs
7. Document and present technical solutions to stakeholders

Part 2: Team Formation and Company Selection

2.1 Team Composition

Aspect	Details
Minimum Team Size	2 members
Maximum Team Size	5 members
Recommended Size	3-4 members for optimal collaboration
Team Roles	Project Manager, Business Architect, Data Architect, Application Architect, Technology Architect (roles can be shared in smaller teams)

2.2 Team Roles and Responsibilities

Role	Primary Responsibilities
Project Manager	Coordinate team activities, manage timeline, ensure deliverables complete, serve as primary contact with instructor
Business Architect	Lead business capability modeling, value stream mapping, stakeholder analysis, goal-to-requirement traceability
Data Architect	Lead enterprise data modeling, system mapping, data flow diagrams, governance framework, polyglot persistence design
Application Architect	Lead C4 model development (Context, Container, Component diagrams), API design, integration patterns
Technology Architect	Lead technology stack selection, deployment architecture, network security, cost estimation, infrastructure as code

2.3 Company Selection Guidelines

Your team must create a hypothetical company that is realistic enough to support detailed architectural analysis. The company should have legitimate IT asset management needs that justify a transformation project.

Required Company Profile Elements

Element	Description	Example
Company Name	Professional, memorable	"Nexus Health Systems," "Apex Financial Group"

Element	Description	Example
Industry	Specific sector	Healthcare, Finance, Education, Retail, Manufacturing, Logistics
Size	Number of employees, locations	500 employees across 3 offices
IT Asset Profile	Types and quantities of assets	800 laptops, 200 mobile devices, 50 servers
Current Pain Points	Specific problems with existing processes	Asset loss, manual tracking, compliance issues
Business Goals	What they want to achieve	Reduce costs, improve compliance, enable remote work

Sample Company Profiles for Inspiration

Example 1: MedHealth Philippines

Aspect	Details
Industry	Healthcare (private hospital network)
Size	1,200 employees across 2 hospitals and 5 clinics
IT Assets	900 computers, 300 tablets for nurses, medical equipment

Aspect	Details
Pain Points	Regulatory compliance, asset tracking across multiple facilities
Business Goals	Achieve 100% audit compliance, reduce asset loss by 50%

Example 2: Skyline University

Aspect	Details
Industry	Higher Education
Size	3,500 employees, 15,000 students, 8 campuses
IT Assets	5,000 faculty/staff computers, 8,000 student lab computers
Pain Points	Seasonal check-in/checkout, student device loans
Business Goals	Reduce checkout time from 15 to 2 minutes, improve utilization

Example 3: Manila Logistics Corp

Aspect	Details
Industry	Logistics and Supply Chain
Size	800 employees, 12 warehouses, 5 regional offices

Aspect	Details
IT Assets	600 rugged laptops, 400 handheld scanners, vehicles
Pain Points	Asset loss in transit, maintenance tracking
Business Goals	Reduce asset loss by 75%, implement predictive maintenance

---

Part 3: Solution Selection and Approval Process

3.1 Option A: Downloadable Software (Customization Required)

You may select an existing open-source software as your foundation and propose customizations to address your company's specific needs.

Approved Software Options

Software	Purpose	Customization Potential	Primary Tech Stack
Snipe-IT	IT Asset Management	High	PHP, MySQL, REST API
OS Ticket	IT Service Management	High	PHP, MySQL, plugin system
Odoo	Enterprise Resource Planning	Very High	Python, PostgreSQL, modular
OrangeHRM	Human Resource Management	Medium	PHP, MySQL

Software	Purpose	Customization Potential	Primary Tech Stack
GLPI	IT Asset & Service Management	High	PHP, MySQL

Required Customization Documentation

Customization Area	Required Details
Polyglot Persistence	Which additional databases? (MongoDB, InfluxDB, Neo4j)
API Development	What new APIs? What existing APIs extended?
AI/ML Integration	What ML capabilities? (demand forecasting, anomaly detection, predictive maintenance)
Integration Points	What external systems? (HRIS, procurement, finance, MDM)
User Interface	What new interfaces? (mobile app, dashboard, self-service portal)

3.2 Option B: Custom Development from Scratch

You may propose building a completely new system. This option gives maximum flexibility but requires more justification.

Required Justification for Custom Development

Justification Element	Description
Gap Analysis	What critical features do existing solutions lack?
Unique Requirements	What makes your company's needs unique?
Strategic Advantage	How does custom development provide competitive advantage?
Total Cost of Ownership	How does custom compare to commercial alternatives?

Technology Stack Options for Custom Development

Tier	Options
Frontend	React, Angular, Vue.js, Flutter (mobile)
Backend	Python/Django, Python/FastAPI, Node.js, Java/Spring, Go
Database	PostgreSQL, MySQL, MongoDB, Cassandra, InfluxDB, Neo4j
Infrastructure	AWS, Azure, GCP, Docker, Kubernetes

Tier	Options
AI/ML	TensorFlow, PyTorch, Scikit-learn, SageMaker, Vertex AI

3.3 Customization/Development Complexity Tiers

Tier	Description	Customizations	Polyglot DBs	AI/ML	Code Lines	Demo Length
Tier 1 (Basic)	Minimum passing	2-3	1 additional	Pre-trained API	500-1,000	10-12 min
Tier 2 (Intermediate)	Good	4-5	2 additional	Custom simple model	1,000-2,000	12-15 min
Tier 3 (Advanced)	Excellent	6+	3+ with CDC	Multiple/deep learning	2,000+	15-20 min

3.4 Pre-Approval Requirements

Before your team can proceed, you must submit a **Project Proposal** for instructor approval.

Project Proposal Template

Section	Required Content
Company Profile	Name, industry, size, location
Problem Statement	2-3 paragraphs describing current challenges

Section	Required Content
Proposed Solution	Build vs. customize decision with justification
Software Selection	If customizing: which software? What customizations?
Key Technologies	Polyglot persistence, AI/ML, APIs to be used
Team Members	Names and assigned roles
Timeline	Proposed completion dates for each deliverable
Complexity Tier	1, 2, or 3 with justification

Approval Criteria

Criteria	Description
Complexity	Is the project sufficiently complex for a capstone?
Feasibility	Can the proposed solution be realistically implemented?
Innovation	Does it incorporate modern technologies appropriately?
Clarity	Is the problem and solution clearly articulated?
Completeness	Are all required elements present?

Part 4: Required Deliverables

4.1 Deliverable Summary Table

Deliverable	Weight	Due	Description
Project Proposal	5%	Week 2	Approval document
Company Background	5%	Week 3	Profile, AS-IS, business case
TO-BE Business Architecture	10%	Week 5	Capabilities, value streams, BPMN
TO-BE Data Architecture	10%	Week 7	Enterprise model, polyglot, DFDs
TO-BE Application Architecture	10%	Week 9	C4 model (Levels 1-3), ADRs
TO-BE Technology Architecture	10%	Week 11	Deployment, stack, cost
Integration Architecture	5%	Week 12	APIs, events, integrations
Implementation Roadmap	5%	Week 13	Phases, WBS, risks
Customization/Development	**15%**	**Week 14**	Working code, customizations

Deliverable	Weight	Due	Description
System Demonstration	**15%**	**Week 14**	Live/recorded demo
Source Code & Documentation	5%	Week 14	GitHub, deployment guide
Executive Presentation	5%	Week 14	12-15 slide deck

4.2 Deliverable 1: Company Background and Problem Statement

Format: Professional document, 3-5 pages

Content Requirements:

Section	Content
Company Profile	Name, industry, size, locations, organizational structure, current IT environment
Current State Assessment (AS-IS)	Process descriptions, pain points (minimum 10 with impact), technology landscape
Business Case	Strategic objectives, quantified benefits, cost of inaction
Solution Vision	High-level description, key capabilities, success criteria (KPIs)

4.3 Deliverable 2: TO-BE Business Architecture

Format: Professional document with BPMN diagrams, 8-10 pages

Content Requirements:

Section	Content
Business Capability Model	Capability map (Levels 1-3), heat map, gap analysis
Value Streams	Minimum 3 value streams with triggers, stages, outputs, metrics
TO-BE BPMN Diagrams	All 6 core processes with swimlanes and automated steps marked
Stakeholder Analysis	Stakeholder register, concerns matrix, RACI charts
Goal-Requirement Traceability	Traceability matrix linking goals to technical requirements

4.4 Deliverable 3: TO-BE Data Architecture

Format: Professional document with diagrams, 8-10 pages

Content Requirements:

Section	Content
Enterprise Data Model	Core entities (minimum 8), ERD, attribute catalog, business rules

Section	Content
Polyglot Persistence Design	Database selection justification, architecture diagram, data-to-database mapping
System Data Mapping	Software table mapping, field mapping, gap analysis, custom fields
Data Flow Diagrams	Level 0 (Context) and Level 1 (Decomposed) DFDs
Data Governance	Ownership matrix, policies, quality metrics, escalation path
Data Synchronization	CDC design, event schemas, sync strategy

4.5 Deliverable 4: TO-BE Application Architecture (C4 Model)

Format: Professional document with C4 diagrams, 10-12 pages

Content Requirements:

Section	Content
Level 1: Context Diagram	System boundaries, user actors (min 8), external systems (min 8), data flows
Level 2: Container Diagram	Technology containers (min 12), communication protocols, descriptions
Level 3: Component Diagram	API application decomposition, layered architecture, interaction flows (min 3)

Section	Content
Level 4: Code (Optional)	UML class diagrams, sequence diagrams for critical components
Architecture Decision Records	Minimum 5 ADRs with context, decision, alternatives, consequences
API Design	RESTful endpoints or GraphQL schema, security design

4.6 Deliverable 5: TO-BE Technology Architecture

Format: Professional document with diagrams, 8-10 pages

Content Requirements:

Section	Content
Deployment Architecture	Container-to-infrastructure mapping, geographic distribution
Technology Stack Matrix	Minimum 20 components with versions, justifications, alternatives
Network Security	VPC/subnet design, security group rules, encryption strategy
Scalability and HA	Scaling strategies, HA configuration, RTO/RPO, DR plan
AI/ML Infrastructure	Training pipeline, serving architecture, monitoring strategy

Section	Content
Cost Estimation	Monthly/annual breakdown, optimization strategies, TCO
Infrastructure as Code	Terraform/Pulumi strategy, environment separation

4.7 Deliverable 6: Integration Architecture

Format: Professional document with diagrams, 5-7 pages

Content Requirements:

Section	Content
API-Led Connectivity	System APIs, Process APIs, Experience APIs
Event-Driven Architecture	Event catalog (min 10 types), topic design, event schemas
External System Integrations	HRIS, procurement, finance, MDM, vendor portals
GraphQL Integration	Schema design, resolver strategy, federation (if applicable)
Integration Testing	Contract testing (Pact), integration test approach

4.8 Deliverable 7: Implementation Roadmap

Format: Professional document, 4-6 pages

Content Requirements:

Section	Content
Phased Approach	4 phases with durations, key deliverables
Work Breakdown Structure	Major tasks, dependencies, estimated effort
Resource Plan	Team composition, skills required, training needs
Risk Management	Key risks, mitigation strategies, contingency plans
Success Metrics	KPIs by phase, baselines, targets

4.9 Deliverable 8: Customization/Development Implementation

Format: Working software, source code repository, deployment guide

Content Requirements:

Section	Content
Customization Scope	Selected software, complexity tier, customizations implemented
Source Code Repository	GitHub/GitLab link, branch structure, commit history

Section	Content
Deployment Guide	Prerequisites, installation steps, environment variables
API Documentation	Swagger/OpenAPI link, Postman collection
Testing Report	Unit test coverage, integration test results, known issues

4.10 Deliverable 9: System Demonstration

Format: Live or recorded video (15-20 minutes)

Content Requirements:

Segment	Duration	Content
Introduction	1-2 min	Problem and solution overview
Architecture Walkthrough	3-4 min	Key diagrams and how they guided implementation
Polyglot Persistence Demo	3-4 min	Multiple databases working together
AI/ML Feature Demo	3-4 min	Model predictions in action
API Demo	2-3 min	Endpoints via Postman/Swagger
Integration Demo	2-3 min	External system connections
User Interface Demo	3-4 min	Key user journeys

Segment	Duration	Content
Conclusion	1-2 min	Achievements and lessons learned

4.11 Deliverable 10: Executive Presentation

Format: Slide deck, 12-15 slides

Slide Content:

Slide	Topic
1	Title slide
2	Executive summary
3	Company background and pain points
4	Solution vision
5	Business architecture highlights
6	Data architecture (polyglot persistence)
7	Application architecture (C4 Level 1 & 2)
8	Application architecture (C4 Level 3 & ADRs)
9	Technology architecture

10	AI/ML integration
11	Integration architecture
12	Implementation roadmap
13	Demo highlights
14	Cost summary and ROI
15	Q&A

Part 5: Technical Resources and Support

5.1 Free Cloud Resources for Students

Service	Free Tier	Use For
GitHub	Unlimited public/private repos	Source code hosting
MongoDB Atlas	512MB storage	Document database
Neo4j Aura	Free instance (limited)	Graph database
InfluxDB Cloud	Free tier (limited)	Time-series database
Redis Cloud	30MB storage	Caching
Supabase	500MB database	PostgreSQL + Auth

Service	Free Tier	Use For
Render	Free backend hosting	API deployment
Vercel	Free frontend hosting	Web app deployment
PythonAnywhere	Free tier	Python backend
Google Colab	Free GPU	AI/ML model training
Hugging Face	Free inference API	Pre-trained models

5.2 Sample Technology Stacks by Complexity Tier

Tier 1 (Basic) Stack

Component	Technology
Frontend	React.js + Tailwind CSS
Backend	Python/FastAPI
Primary DB	PostgreSQL (Supabase free tier)
Polyglot DB	MongoDB Atlas (free tier)
AI/ML	Hugging Face Inference API
Deployment	Render + Vercel (free tiers)

Tier 2 (Intermediate) Stack

Component	Technology
Frontend	React.js + Material UI
Backend	Python/FastAPI + Celery
Primary DB	PostgreSQL (Neon free tier)
Polyglot DBs	MongoDB Atlas + Redis Cloud
AI/ML	Scikit-learn model + FastAPI serving
Message Queue	Redis (for Celery)
Deployment	Railway + Vercel

Tier 3 (Advanced) Stack

Component	Technology
Frontend	Next.js + Tailwind CSS
Backend	Python/FastAPI + Temporal
Primary DB	PostgreSQL (TimescaleDB extension)
Polyglot DBs	MongoDB Atlas + InfluxDB + Neo4j Aura
AI/ML	TensorFlow/PyTorch + SageMaker

CDC	Debezium + Kafka (Redpanda free tier)
Search	Elasticsearch (Bonsai free tier)
Deployment	AWS free tier / Google Cloud free credits

Part 6: Grading Rubric

6.1 Overall Grade Breakdown

Deliverable	Weight
Project Proposal	5%
Company Background	5%
TO-BE Business Architecture	10%
TO-BE Data Architecture	10%
TO-BE Application Architecture	10%
TO-BE Technology Architecture	10%
Integration Architecture	5%
Implementation Roadmap	5%
Customization/Development (Code)	15%

Deliverable	Weight
System Demonstration (Live/Video)	15%
Source Code & Documentation	5%
Executive Presentation	5%
Total	**100%**

6.2 Detailed Rubric for Each Deliverable

Criterion	Weight	Excellent (90-100%)	Satisfactory (70-89%)	Needs Improvement (<70%)
Completeness	25%	All required sections present, detailed	Most sections present	Missing major sections
Quality of Analysis	25%	Insightful, thorough, well-reasoned	Good analysis, some gaps	Superficial or incorrect
Use of Modern Technologies	20%	Excellent integration of polyglot, AI/ML, APIs	Good use, some gaps	Minimal or inappropriate

Criterion	Weight	Excellent (90-100%)	Satisfactory (70-89%)	Needs Improvement (<70%)
Traceability	15%	Clear links from pain points to solutions	Some traceability	No traceability
Professional Presentation	15%	Professional formatting, clear diagrams, no errors	Good formatting, minor issues	Poor formatting, many errors

6.3 Demonstration Scoring Criteria

Criteria	Weight	Excellent	Satisfactory	Needs Improvement
System Functionality	30%	All promised features work	Most features work	Major features broken
Polyglot Persistence	20%	Multiple DBs working seamlessly	Two DBs working	Single DB only
AI/ML Integration	20%	Model works, produces	Model works but limited value	No AI or not working

Criteria	Weight	Excellent	Satisfactory	Needs Improvement
		valuable predictions		
API Quality	10%	Well-documented, RESTful, auth	Basic API, some issues	No API or non-functional
Presentation Quality	10%	Professional, clear, well-paced	Acceptable	Poorly organized
Alignment with Architecture	10%	Implementation matches diagrams	Mostly matches	Significant deviation

---

Part 7: Sample Project Scenarios

7.1 Scenario 1: Snipe-IT Customization for University

Aspect	Details
Company	Skyline University (15,000 students, 3,500 staff)
Software	Snipe-IT (customized)
Customizations	1. MongoDB for student device loan history 2. React Native mobile app for scanning 3. REST API for SIS integration

Aspect	Details
	4. Custom dashboard for department chairs 5. Automated offboarding workflow
AI Feature	Demand forecasting for lab computer usage
Polyglot	PostgreSQL (core) + MongoDB (loan history) + Redis (caching)
Complexity Tier	Tier 2 (Intermediate)

7.2 Scenario 2: Custom System for Logistics

Aspect	Details
Company	Manila Logistics Corp (800 employees, 12 warehouses)
Software	Custom development (Python/React)
Features	1. Asset tracking with QR codes 2. Real-time telemetry ingestion 3. Predictive maintenance alerts 4. Graph-based network mapping 5. Mobile app for warehouse staff
AI Feature	Predictive maintenance for warehouse equipment
Polyglot	PostgreSQL (core) + InfluxDB (telemetry) + Neo4j (relationships)

Aspect	Details
Complexity Tier	Tier 3 (Advanced)

7.3 Scenario 3: Odoo Customization for Healthcare

Aspect	Details
Company	MedHealth Philippines (1,200 employees, 2 hospitals)
Software	Odoo (Inventory + Maintenance modules)
Customizations	1. HIPAA-compliant asset tracking 2. Integration with nurse call system 3. Automated warranty claims to vendors 4. Compliance reporting dashboard 5. RFID integration for high-value equipment
AI Feature	Anomaly detection for equipment usage patterns
Polyglot	PostgreSQL (Odoo core) + TimescaleDB (equipment telemetry)
Complexity Tier	Tier 2 (Intermediate)

Part 8: Collaboration and Submission Guidelines

8.1 Team Collaboration Best Practices

Practice	Recommendation
Regular Meetings	At least once per week (synchronous)
Communication Channel	Dedicated Slack/Discord/Teams channel
Document Collaboration	Google Docs or SharePoint
Code Collaboration	GitHub with branches, pull requests, issues
Version Control	Use consistent naming conventions

8.2 Conflict Resolution Process

1. **First step:** Discuss within team to find resolution
2. **Second step:** Bring to instructor for mediation
3. **Third step:** Individual contributions assessed separately if necessary

8.3 Peer Evaluation

At the end of the project, each team member submits a confidential peer evaluation. Significant discrepancies in contribution will be reflected in individual grades.

Peer Evaluation Template:

Team Member	Self Rating (1-10)	Peer Rating (1-10)	Contribution Comments
[Name 1]	/10	/10	[Comments]
[Name 2]	/10	/10	[Comments]

8.4 Submission Instructions

1. Create a team GitHub repository
2. Upload all documentation files (PDF format)
3. Upload source code with README.md
4. Upload demonstration video (YouTube unlisted or Google Drive)
5. Submit repository link via course LMS
6. Present live to instructor on scheduled date

Part 9: Frequently Asked Questions

Q: Do we actually have to deploy the system or just run it locally?

A: You must demonstrate the system running. This can be locally on your machine during the live demo, or deployed to a cloud platform. Deployment is encouraged but not strictly required.

Q: Can we use free cloud credits for deployment?

A: Yes! AWS, Azure, and GCP offer free credits for students through GitHub Education. MongoDB Atlas, Neo4j Aura, and InfluxDB Cloud also have generous free tiers.

Q: What if our AI model doesn't work perfectly?

A: The goal is to demonstrate integration, not perfect accuracy. A model that works reasonably well (even with limitations) is acceptable. Document the limitations.

Q: Can we use pre-trained models or APIs instead of training our own?

A: Yes. Using Hugging Face models, OpenAI API, or cloud AI services (SageMaker, Vertex AI) is perfectly acceptable and often more practical.

Q: How much of the system needs to be working for the demo?

A: At minimum, the core user journey (e.g., assign asset → return asset → search asset) must work. Polyglot persistence and AI features must also be demonstrable.

Q: Can we work on the code as a team using GitHub?

A: Yes, this is required. Use branches, pull requests, and issues to manage collaboration. Your GitHub history will be reviewed.

Q: What if our team has unequal coding skills?

A: Assign roles based on strengths. Team members with less coding experience can focus on architecture documentation, testing, UI design, or project management.

Q: Can we use low-code/no-code platforms?

A: No. The purpose is to demonstrate architectural and implementation skills. Low-code platforms bypass the learning objectives.

Q: What is the minimum viable product for custom development?

A: Asset CRUD, assignment functionality, basic search, REST API, one database (PostgreSQL), polyglot extension (MongoDB), one AI feature, basic web frontend.

Q: Can we change our company or solution after approval?

A: Minor changes are acceptable with instructor approval. Major changes require resubmission of the proposal.

Part 10: Project Timeline (6 Weeks)

Week	Deliverable	Activities
Week 1*(4-25)*	Team Formation	Form teams, choose company, draft proposal
Week 1	Project Proposal	Submit for approval, revise as needed
Week 1	Company Background	Research, document AS-IS, identify pain points
Week 2 *(5-2)*	Business Architecture	Develop TO-BE capability map, value streams, BPMN
Week 2	Business Architecture Due	Refine, finalize, submit
Week 2	Data Architecture	Enterprise model, polyglot design, DFDs
Week 2	Data Architecture Due	Refine, finalize, submit
Week 3 *(5-9)*	Application Architecture	C4 models, ADRs, API design
Week 3	Application Architecture Due	Refine, finalize, submit
Week 4 *(5-16)*	Technology Architecture	Deployment, stack, security, cost

Week	Deliverable	Activities
Week 4	Technology Architecture Due	Refine, finalize, submit
Week 5 *(5-23)*	Integration + Roadmap	Integration patterns, implementation plan
Week 5	Integration + Roadmap Due	Refine, finalize, submit
Week 6*(5-30 to 6-13)*	Final Deliverables	Code, demo, presentation, all documentation

---

Part 11: Conclusion and Success Factors

11.1 Key Success Factors

1. **Start Early** - The project requires significant time for both design and implementation
2. **Choose Appropriate Scope** - Select a complexity tier that matches your team's skills
3. **Trace Everything** - Every decision should link back to a pain point or business goal
4. **Use Version Control** - GitHub is required; use branches and pull requests
5. **Test Continuously** - Don't wait until the end to test your implementation
6. **Document as You Go** - Don't leave documentation until the last week
7. **Practice Your Demo** - The demonstration is 15% of your grade
8. **Communicate Regularly** - Weekly meetings are essential for team alignment

11.2 What Makes an Excellent Project

Aspect	Excellent Project Characteristics
Completeness	All deliverables fully completed with attention to detail
Integration	Polyglot databases work together seamlessly; AI features add real value
Traceability	Clear line from pain points → requirements → architecture → implementation
Professionalism	Documentation is polished; diagrams are clear; demo is rehearsed
Innovation	Creative use of modern technologies beyond minimum requirements
Resilience	System handles errors gracefully; fallbacks implemented

11.3 Final Reminders

1. **Seek approval before coding** - Your proposal must be approved first
2. **Polyglot persistence is required** - At least one additional database beyond primary
3. **AI/ML integration is required** - At least one working ML feature
4. **Working demo is required** - Live or recorded video (15-20 minutes)
5. **Source code must be shared** - GitHub repository with README
6. **All team members must contribute** - Peer evaluation will be used

ASDI Group Project: Comprehensive Project Documentation Contents

Outline Only Version)

Document Structure Overview

```
ASDI_Group_Project_Documentation/
│
├── 00_Cover_Page/
│   ├── Cover_Page
│   └── Table_of_Contents
│
├── 01_Executive_Summary/
│   └── Executive_Summary
│
├── 02_Company_Background/
│   ├── Company_Profile
│   ├── ASIS_Assessment
│   └── Business_Case
│
├── 03_TOBE_Business_Architecture/
│   ├── Business_Capability_Model
│   ├── Value_Streams
│   ├── BPMN_Diagrams
│   ├── Stakeholder_Analysis
│   └── Goal_Requirement_Traceability
│
├── 04_TOBE_Data_Architecture/
│   ├── Enterprise_Data_Model
│   ├── Polyglot_Persistence_Design
│   ├── System_Data_Mapping
│   ├── Data_Flow_Diagrams
│   ├── Data_Governance
```

```
│   └── Data_Synchronization_Strategy
│
├── 05_TOBE_Application_Architecture/
│   ├── C4_Level1_Context_Diagram
│   ├── C4_Level2_Container_Diagram
│   ├── C4_Level3_Component_Diagram
│   ├── C4_Level4_Code_Diagrams
│   ├── Architecture_Decision_Records
│   └── API_Design
│
├── 06_TOBE_Technology_Architecture/
│   ├── Deployment_Architecture
│   ├── Technology_Stack_Matrix
│   ├── Network_Security_Architecture
│   ├── Scalability_and_HA
│   ├── AI_ML_Infrastructure
│   ├── Cost_Estimation
│   └── Infrastructure_as_Code
│
├── 07_Integration_Architecture/
│   ├── API_Led_Connectivity
│   ├── Event_Driven_Architecture
│   ├── External_System_Integrations
│   ├── GraphQL_Integration
│   └── Integration_Testing_Strategy
│
├── 08_Implementation_Roadmap/
│   ├── Phased_Approach
│   ├── Work_Breakdown_Structure
│   ├── Resource_Plan
│   ├── Risk_Management
│   └── Success_Metrics
│
├── 09_Customization_Development/
│   ├── Customization_Scope
│   ├── Source_Code_Repository
│   ├── Deployment_Guide
│   ├── API_Documentation
│   └── Testing_Report
```

```
│
├── 10_Demonstration/
│   ├── Demo_Script
│   ├── Demo_Video_Link
│   └── Demo_Screenshots
│
├── 11_Appendices/
│   ├── Glossary
│   ├── References
│   ├── Peer_Evaluation
│   └── Additional_Materials
│
└── 12_Presentation/
    ├── Final_Presentation_Slides
    └── Presentation_Notes
```

Prepared for: ASDI - Advanced Systems Design and Implementation